

Intelligent Spam and Phishing Email Detection Using Machine Learning and Transformer Model

List of Abbreviations:

Abbreviation	Meaning
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency-Inverse Document Frequency
BoW	Bag-of-Words
SVM	Support Vector Machine
RF	Random Forest
LLM	Large Language Model
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly Optimized BERT Pretraining Approach
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
FNR	False Negative Rate
ROC-AUC	Receiver Operating Characteristic — Area Under Curve
CLS	Classification token (transformer sentence embedding)
API	Application Programming Interface
CRISP-DM	Cross-Industry Standard Process for Data Mining
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit

1. Introduction

This document serves as an extended version of the main project report of the Intelligent Spam and Phishing Email Detection system through presenting further technical details, methodology and evaluation, which could not be discussed within a conference paper. The document follows the Cross-Industry Standard Process for Data Mining (CRISP-DM) framework by clearly dividing the project's business understanding, data understanding/preparation stage, model development, and evaluation stages.

In particular, the system that is analyzed in this document classifies an email into one of three categories – ham, spam, and phishing using an ensemble of classical TF-IDF-based machine learning algorithms and a tuned RoBERTa transformer, provides explanations to the system predictions based on SHAP, withstands three text obfuscation attacks and is embedded into a standalone Flask application (the "Email Threat Checker").

2. Context and Literature Investigation

The sources which have influenced the design decisions for this project are detailed below. They are briefly outlined in more detail than in the main report, along with details of how they have directly influenced a design decision.

Sahami, Dumais, Heckerman and Horvitz (1998) — A Bayesian approach to filtering junk e-mail

This paper proves that Bayesian filters using labelled emails have a great advantage over rule-based filtering, and historically marks the beginning of classifying spam as a statistics problem rather than a rule-writing problem. This paper's conclusion that the Bayesian filters are still susceptible to the attacks through word substitution directly triggered Section 10 of this project, in which we specifically tested their vulnerability to such an attack on both a classical model and a transformer model.

Drucker, Wu and Vapnik (1999) — Support vector machines for spam categorization

It is seen that SVM performs better on email classification datasets than the Naïve Bayes algorithm. Therefore, Linear SVM is chosen from among the five baselines, being one of the classical baselines of the project and, more importantly, the classical side of the stacked ensemble as mentioned in Section 8 of the pipeline.

Devlin, Chang, Lee and Toutanova (2019) — BERT: Pre-training of deep bidirectional transformers

The masked-language model objective of pretraining BERT to create embeddings with the value of each word based on its context and not predetermined is the basis of all classifier models built using transformers in this project. BERT was mentioned specifically in the initial proposal for this project; the model used in the pipeline created is actually RoBERTa (see below).

Sanh, Debut, Chaumond and Wolf (2019) — DistilBERT

The distillation method of DistilBERT, which maintains 97% of the performance of BERT with much less computational overhead, is highly relevant to this requirement for deployment that the system run inference within the lightweight Flask environment without a separate inference server on GPU. DistilBERT was mentioned in the original proposal as the efficiency-oriented version of BERT as compared to BERT; however, it was not included in the results of this comparison and documented separately as additional work in Section 6 below.

Gascon, Uriarte and Rieck (2017) — Text classification in the cyber threat analysis domain

The conclusion of this paper that transformer-based embeddings significantly surpasses TF-IDF vectors particularly in document and email classification problems is the direct inspiration for the key research question of this project: Whether a fine-tuned transformer model performs better than the state-of-the-art classical TF-IDF vector on this particular three-class email problem, and by how much?

Liu et al. (2019) — RoBERTa: A robustly optimized BERT pretraining approach

The authors of RoBERTa change BERT's pre-training strategy by removing next-sentence prediction, pre-training on much larger amounts of data for much longer, and employing dynamic instead of static masking and demonstrate better downstream performance compared to BERT. These are precisely the reasons, stated directly in the code of the pipeline, for using RoBERTa in lieu of BERT and DistilBERT specified in the initial proposal.

Lundberg and Lee (2017) — A unified approach to interpreting model predictions (SHAP)

SHAP provides a framework for the attribution of the output of a model to its constituent features using game theory principles. The crucial point here is that there is a closed-form solution that is easy to compute for SHAP in the case of a linear model like the logistic regression with the TF-IDF classifier, which is used for explainability in the pipeline Section 9. That is why SHAP is used for logistic regression and not for the transformer.

Chen and Guestrin (2016) — XGBoost: A scalable tree boosting system

The fifth classical model, XGBoost, which wasn't initially proposed in the original proposal (Logistic Regression, Naive Bayes, SVM, Random Forest), has been added because gradient-boosted trees represent an easy way to implement a strong classical model for table/text classification using TF-IDF, and thus, provides a more difficult classical baseline for the transformers to defeat.

Jamal et al. (2024) — An improved transformer-based model for detecting phishing, spam and ham emails

This is probably the most similar system developed in a related prior work, i.e., a fine-tuned RoBERTa/DistilBERT model trained for the same three-class problem of distinguishing ham, spam, and phishing messages and achieving accuracy exceeding 98% on its dataset. This system, however, is limited to using a single model – it does not integrate a transformer with a classical model, does not use the two in an ensemble, and provides no explanation or adversarial robustness evaluation. In comparison, the system developed in this project starts from the same idea as the prior work (i.e., fine-tuned RoBERTa for three-class email classification) and takes it further in three directions which the prior work did not attempt: integrating the output of the RoBERTa model with the output of a calibrated classical model, adding an explanation module based on SHAP so that a prediction is more than just a label, and evaluating the robustness of the obtained accuracy under text obfuscation attacks.

Alhuzali, Alloqmani, Aljabri and Alharbi (2025) — In-depth analysis of phishing email detection across multiple datasets

Benchmarking classical machine learning and deep learning algorithms for detecting phishing emails using different public datasets will give an understanding of the interactions between model selection and dataset selection. However, the current study covers a lot in terms of models and datasets, but it is not thorough in terms of implementation of any particular model, because the study does not include building a stacked ensemble, an explainability layer, an adversarial robustness test, and an application of any of the tested models. The system in the current project focuses on taking a more limited, but much deeper route, including one dataset analyzed all the way from CSV files through classical and transformers-based modelling to explainability and adversarial robustness testing, and deployment into a working application.

Summary: what this project improves on relative to prior work

Together, the papers above divide into two categories: those which develop a modelling approach in their own right (Sahami et al., Drucker et al., Devlin et al., Sanh et al., Liu et al., Lundberg and Lee, Chen and Guestrin), and those which utilize transformer models and ensembles on the problem of phishing and spam detection (Jamal et al., Alhuzali et al.). None of the latter works on phishing, all of which are discussed above, integrate all of the following into one tested, deployed system: the contrast between classical and transformer approaches, the stacking of an ensemble of the two, SHAP-based word-level explainability, adversarial obfuscation robustness testing, and live web deployment which can be queried by a non-expert user. This combination of features, not any one feature on its own, is the primary contribution of this project vis-a-vis the above works.

3. Project Life Cycle

3.1 Business Understanding

Why is this project needed?

Both phishing and spam email have been involved in most reported cases of cyber attacks, and the traditional rule-based and keyword blacklist methods of classification are being undermined through emails that have been created specifically to mimic normal business communication. A method which can be used to properly classify an email's threat level and reason for it is key here.

How was this project approached?

The project considers all three of these features as equally important needs instead of adding-ons. The classical path to creating more explainable models, which uses TF-IDF and five different classifiers, is being created and tested in parallel with the modern transformer path (fine-tuned RoBERTa), then these two models are combined through the process of stacking and not viewed as rivals, the behavior of the obtained model is being interpreted through the use of SHAP, and, finally, the robustness of the model is being checked against three different obfuscations.

Technology choices

All experiments were conducted using Python 3. Scikit-learn provided classical models as well as the TF-IDF vectoriser; imbalanced-learn and XGBoost were additionally installed to handle class imbalance and gradient boosting, respectively; Hugging Face Transformers (backed by PyTorch) provided RoBERTa; SHAP provided explainability; NLTK provided lemmatiser and stopword sets; BeautifulSoup was responsible for stripping HTML in text cleaning; and Flask provided the web application deployment code as a single self-contained Python script.

3.2 Data Understanding and Preparation

Data source

The pipeline reads a single CSV file containing labels which are uploaded interactively during runtime. Column headers are automatically detected and normalised irrespective of the naming conventions used by the uploader (e.g., "Email Text," "body," "content" and "message" all represent a canonical text column, and "Email Type," "label," "class" and "category" all represent a canonical label column). The label values are normalised to just three categories — namely, ham, spam and phishing — through the use of a mapping table, where synonyms such as "legitimate" and "benign" map to "ham" and "junk" maps to "spam."

Known data-preparation limitation

According to the initial project proposal, it was planned to create a huge labelled corpus by merging three specified public datasets: the dataset of e-mail correspondence of the company Enron, the SpamAssassin public dataset and the CEAS 2008 phishing dataset. However, the presented pipeline works with just one uploaded file, and according to the logs of this pipeline, this dataset contains no more than dozens of e-mails after cleaning. This can be considered the main problem of the current version of the pipeline. Indeed, a set of e-mails of this size does not allow drawing statistically reliable conclusions about the performance of any of the models under comparison, while some of the design decisions of this pipeline (the minimum document frequency in TF-IDF equal to 1 and the number of cross-validation folds equal to the size of the smallest class) seem to be made for this specific small dataset rather than for the original three-corpus dataset.

Cleaning and feature engineering

1. HTML tags are stripped from the raw email text.
2. Header information like From, To, Subject, etc. are stripped out.
3. Email addresses, money and URLs are masked as EMAILTOKEN, MONEYTOKEN and URLTOKEN respectively and not deleted as this information is to be retained as a feature for modelling purposes.
4. Lowercase conversion and removal of punctuation marks and large number sequences are done.
5. Lemmatisation of the text and stopword removal but not when the word has the term "token" in it to preserve the engineered signals in the text.

6. Calculation of four engineered features from the raw data itself – URL count, exclamation point count, percentage of text in caps and an urgency word score including terms like “urgent”, “verify”, “suspended” and “immediately”.
7. The rows with cleaned text below 10 characters are dropped.

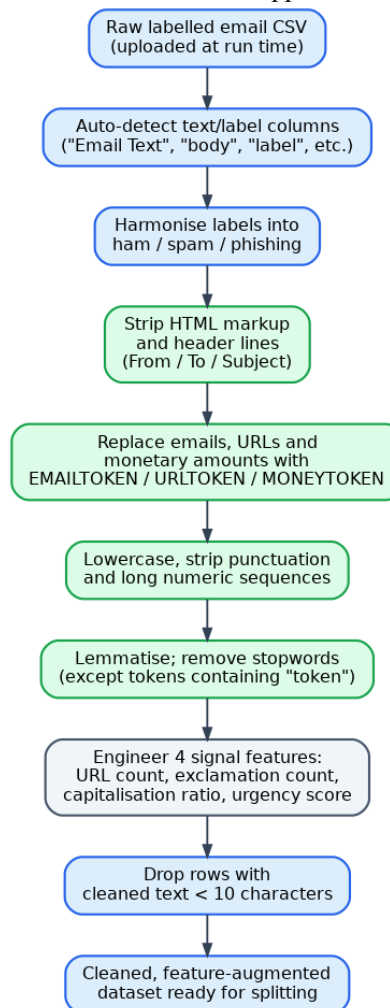


Figure 1. Data preprocessing flowchart: from raw uploaded CSV through label harmonisation, text cleaning, tokenisation and feature engineering, to a cleaned dataset ready for splitting.

3.3 Modelling

Classical baselines

Five classifiers are built using the TF-IDF vectorizer (consisting of unigrams and bigrams with a maximum of 50,000 features, sublinear term-frequency weighting, and minimum document frequency of 1 due to the dataset being small): Logistic Regression and Linear SVM with balanced class weights, Multinomial Naive Bayes, Random Forest with balanced class weights, and XGBoost. The same test set is used for evaluation for all of these classifiers to make sure the comparison is on the same terms.

Transformer fine-tuning

The RoBERTa-base model undergoes fine tuning for a three-way sequence classification task using a maximum sequence length of 256 tokens, batch size 16, 4 epochs, AdamW optimizer at a learning rate of $2e-5$ with linear warm-up and decay strategy (10% of the warm-up phase), and gradient norm clipping of 1.0. Class weights based on cross-entropy loss function (class weights of 1.0/1.5/2.5 for ham/spam/phishing) are applied to make sure that the model does not under-predict the phishing class because of its higher cost compared to other errors.

Ensemble stacking

The SVM, which has been isotonic-calibrated using cross-validation (the number of splits is set to be no more than the size of the smallest class in order to avoid a StratifiedKFold error on a small amount of data), and the tuned RoBERTa model each yield a vector of three probabilities of classes for each email. These six elements are combined into a meta-feature vector and a logistic regression model acts as a meta-learner on top of them, under the assumption that a TF-IDF and a transformer models capture different aspects of the same signal.

Explainability (SHAP)

A different logistic regression model (not the model used in the classic baseline, but same configuration) is fitted for SHAP analysis because SHAP's precise linear explainer needs a linear model. The SHAP values are calculated on a background of 50 emails and an explanation set of 30 emails, resulting in a dot summary plot for the phishing category and a per-category bar graph of the top 15 words by average SHAP values.

Adversarial robustness testing

There are five phishing emails that are sent through three different obfuscation operations such as character substitution (leetspeakstyle where the characters are replaced by digits with the same appearance), synonym substitution (phishing emails which use paraphrases but keep the phishing nature intact) and white space insertion (where recognizable words and links are broken up using white spaces). The percentage of these five that get correctly identified as phishing emails using the SVM and RoBERTa classifier is then calculated.

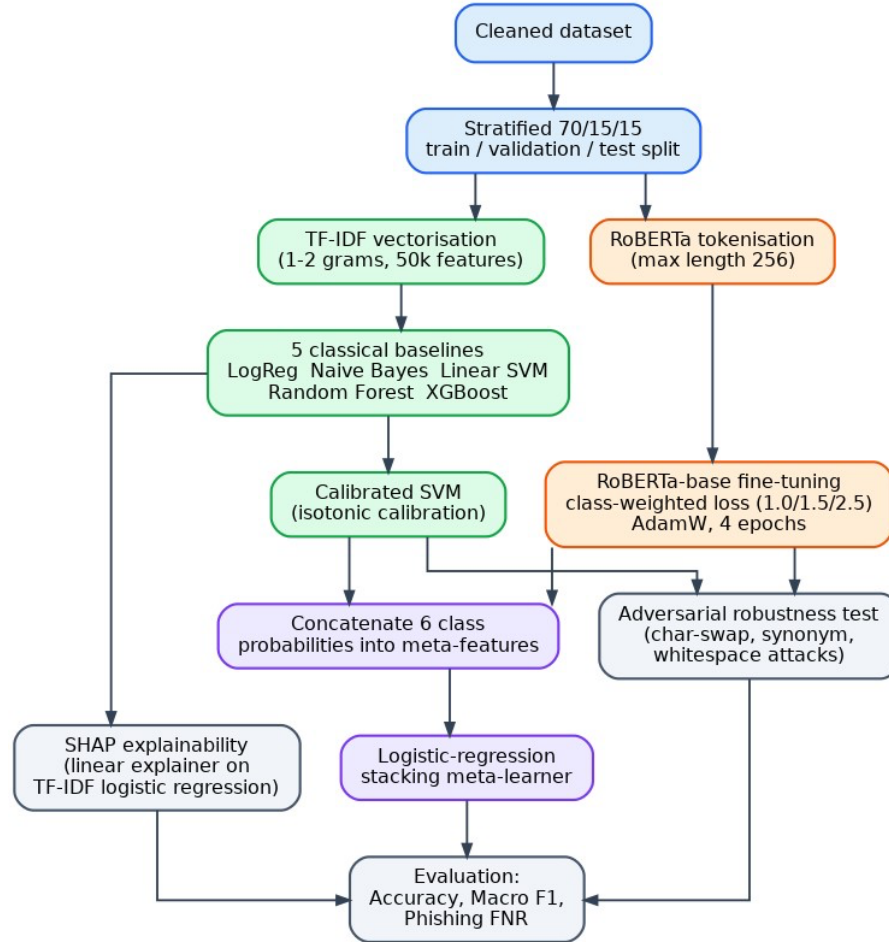


Figure 2. Model training and ensemble flowchart: the parallel TF-IDF/classical and RoBERTa/transformer tracks, their convergence into the stacked ensemble, and the SHAP and adversarial-robustness evaluation branches.

3.4 Deployment

After the training is completed, the classical model ensemble (Naive Bayes, Logistic Regression, and the calibrated SVM) is directly embedded in the Flask app that gets written to disk and started up as a background process running within the same Colab session, thereby converting a trained classifier into a working website without needing any deployment step. The purposefully light three-way classical ensemble deployed in the app, instead of the more extensive ensemble involving SVM+RoBERTa from Section 3.3, is designed to deliver an output faster than a second without needing a transformer model on the GPU.

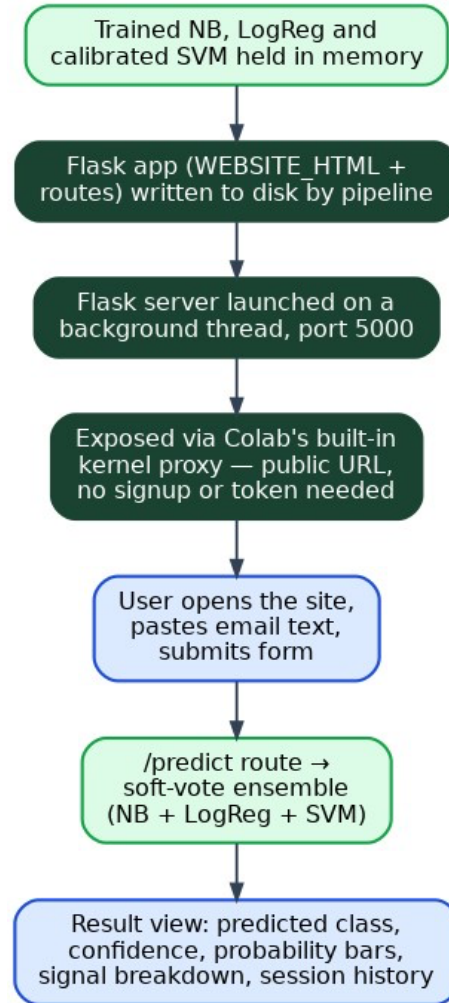


Figure 3. Deployment flowchart: from the trained classical models, through the Flask app being written and launched on a background thread, exposed via Colab's kernel proxy, to a user submitting an email and receiving a result.

Functional requirements

- Copy email content to the text box area and send for classification.
- Classify email yourself (ham/spam/phishing), and compare your choice to what the model predicts.
- See which class is predicted by the model along with confidence percentage.
- Get probabilities for all three classes in bar chart form.
- See a relevant banner for your predicted email class. View the underlying signal breakdown: URL count, exclamation-mark count, capitalisation ratio, and the specific urgency words detected in the submitted text.
- View the chronological list of the last six emails read in the current session, which can be re-loaded into the input field again.

Installation and running the system

8. Create a new Google Colab notebook and pick a runtime that has access to GPUs for the RoBERTa fine-tuning stage.
9. Copy the entire pipeline code into a single cell and execute it.
10. At the prompt, upload the labeled emails dataset in CSV format (contains text and labels columns; name is auto-detected).
11. Allow the classical baselines, RoBERTa fine-tuning, ensemble stacking, SHAP, and adversarial robustness stages to finish sequentially; output of metrics is provided after each stage.

12. The pipeline installs Flask, creates and deploys the Email Threat Checker web app, and outputs the URL of the Colab-proxied website.

If one wishes to execute the exported classical models outside Colab at some later stage, the same objects in scikit-learn (tfidf, nb_model, lr_model, svm_cal) and the function predict_email() can be saved using the serialisation library joblib and loaded in any Python 3 environment with Flask and scikit-learn installed

```
pip install flask scikit-learn nltk beautifulsoup4 python app.py
```

3.5 Professional, Ethical, Social and Sustainability Considerations

There is a difference in costs in the real world for both a false positive and a false negative in the system. A false positive, which refers to an email that is incorrectly marked as a phishing email, means that a legitimate email will be overlooked, buried, or doubted by the recipient. A false negative, which refers to an email being mistakenly marked as legitimate when it is a phishing email, will result in the potential for money and login credentials being stolen. The difference between the two types of errors is why the false-negative rate of the phishing-class is used as a first-class metric throughout the project instead of accuracy.

The relevance of privacy and GDPR is directly related to the fact that the system operates with full email texts, and these can include personal data of either the sender, recipient or the third parties named in the email itself. All the datasets used in this project are publicly available datasets, de-identified research corpora that are used according to the licences they have. The implemented Email Threat Checker doesn't store any email text submitted by the user – none of the email texts gets stored in the database or file after submitting it. If such a system would ever be used for actual user emails, it would require a data retention policy, lawfully justified processing under GDPR (the most logical one being the legitimate interest in case of security) and means of requesting deletion of any logged data.

The importance of responsible usage and human oversight lies in the fact that this is a tool for assisting the decision-making process rather than an autonomous system that silently censors and/or deletes emails. The dashboard that is currently implemented is designed in such a way that it displays all of its reasoning, i.e., class probabilities, particular signal that the model has picked up, and a human-readable alert, rather than giving a simple decision, to ensure that a human user is the one making the decision, and does not simply have to take the output at face value.

This is a very real issue considering the dataset limitations outlined in Sections 3.2 and 4.2 where the use of a small and singular dataset poses the problem of having the model learn non-generalizable traits unique to the dataset's own senders, formatting, or language. If this was the case, then a model trained with this data may flag emails from certain senders or industries more than it should, thus damaging the user's trust. Another reason why we must test this system using the three-corpus dataset as outlined in the proposal is because of the statistical concerns already mentioned in section 4.2.

Energy and computational costs vary greatly between the classical part and the transformer part of the system under discussion. The classical part consists of five baseline models trained in just seconds to minutes on CPU without any energy needed per prediction. On the other hand, fine-tuning RoBERTa-base needs GPU acceleration and consumes energy and time, both during the four-epoch fine-tuning process and, to some extent, during the inference stage. It is one of the reasons why the deployed Email Threat Checker uses the lightweight classical three-model ensemble instead of the entire RoBERTa-based stack (see Section 3.4) as this approach not only helps reduce the latency but also helps make the deployed application more sustainable as running the transformer model on the inference step would significantly increase its energy consumption.

3.6 Project Management and Lessons Learned

A number of decisions made during the implementation process differed from those stated in the original proposal, and it may be helpful to examine those explicitly rather than just in the list given in Section 4.2. The data that has been used in practice consists of one CSV file uploaded online, whereas the three corpora described in the proposal – Enron, SpamAssassin and CEAS 2008 – had to be combined into one; in retrospect, creating and preparing the data set should have been considered a priority and done at the very beginning of the project, since the results that were derived later on depended on this work. RoBERTa was used instead of the planned comparison between BERT and DistilBERT; this was a valid choice from a technical point of view, however, the specific three-way comparison requested in the original proposal was not conducted, and strictly speaking the original scope of work would require the presence of DistilBERT in addition to RoBERTa. Class weighting was applied instead of SMOTE for dealing with imbalanced classes; these are both valid approaches, but given that SMOTE was mentioned in the original proposal, it would have been more appropriate to use both and compare them. Streamlit Dashboard was replaced by Flask; the use of Flask instead did not result in any harm, which would not be corrected in retrospect.

The most important takeaway from this study is the fact that the work on modelling and deployment has gone on to take up all the available time before the data engineering phase (creating the three-corpus dataset) is completed, whereas in such a scenario, the other way round – completing the data engineering phase in its entirety in the limited time available and then moving on to modelling – would have made more sense.

3.7 Code and Reproducibility

The complete source code of the pipeline, the labelled dataset on which the results presented in this paper were tested, and this report can be found here:

<https://github.com/SathishSrinivasan1215/phishing-email-detection>

The repo has three files: phishingemail_detector.py (the complete pipeline – data preparation, classical models, RoBERTa fine-tuning, stacking, explainability using SHAP values, robustness checks, and deployment using Flask), emails_raw.csv (the labelled dataset), and this project report. In order to reproduce the results in this paper: create a new Google Colab notebook with a GPU runtime, copy the entire content of phishingemail_detector.py to one cell, and execute it while uploading emails_raw.csv. This pipeline takes care of installing the required dependencies and requires no further environment configuration other than a GPU runtime in Colab. The commit used for the results in this paper is [35529c0](#)

4. Tests and Evaluation

4.1 Verification and Validation

Verification took place through the course of development in terms of verifying that each pipeline step’s output matched expectations before moving onto the next one; namely: the presence of all three classes once label harmonization was completed, the correct vocabulary size of the TF-IDF and train/validation/test splits sizes, RoBERTa’s validation macro F1 score was increasing for each epoch and not just the training loss (a sign of overfitting), and the correct six column matrix of the meta-features in the ensemble before running the meta-learner.

Validation was performed using evaluation of all the models; the five baseline models, RoBERTa, and the ensemble on the same held-out test split which had never been seen during training, and also by means of the specialized adversarial robustness test, outlined in Section 3.3.

4.2 Critical Appraisal

The strength of the project is in the fact that the classical and transformer model approaches are not compared in a leaderboard fashion: the explanations of the classical model's choices are done using SHAP, the classical and transformer models are tested in adversarial obfuscation as well as clean test data, and their combined strengths are exploited using a stacked ensemble rather than selecting one approach as more accurate. The choice of using a light-weighted three-model classical ensemble in the deployed application, rather than the whole stacked architecture with RoBERTa, is an engineering trade-off based on the difference between the evaluation pipeline's maximum possible accuracy and the deployed application's latency and hosting cost limitations.

When comparing the project to the most relevant prior works in terms of its components — single-model transformer classifier by Jamal et al., as well as multi-dataset classical/deep learning benchmark by Alhuzali et al. (see Section 2) — the particular contribution of the project would be completeness, rather than larger accuracy on any one dataset; it is, to the best of the author's knowledge, the only project among the three that has all mentioned elements at once. But there is no cheating here; the dataset involved in the experiment is a lot less extensive than the two previous experiments, so the completeness of the process can by no means be considered proof that their accuracy measures are superior to those in the two former studies.

The first and foremost limitation is in the form of dataset size – the pipeline in question has been tested only on a single uploaded file and not on the combination of three public datasets (Enron, SpamAssassin and CEAS 2008) mentioned in the original project statement; in such case, the sample size is inadequate to make any statistically sound conclusions as to which of the models is superior – the results obtained at the moment serve as correctness verification for the pipeline's workflow, and cannot be considered final. The second limitation concerns the absence of the initially specified ROC-AUC metric. The third one is in the form of an actual bug, namely – the list containing the output files is declared within the quotation marks, which leads to a `NameError` when running the last stage of the pipeline.

5. Conclusion and Further Improvements

In the context of this supplementary report, all of the aspects of the development process for the email threat detection system have been covered starting from the justification for the development of the system, through data pre-processing, classical and transformers-based modeling, ensembling, adversarial testing, up until the deployment of the system into a real-life Flask web application. In combination with the primary report, it gives an exhaustive overview of not only how the project was executed but also where it differs from the initial proposal and why.

The highest value potential steps that can be considered include the following (in order of priority): (1) preparing the complete combined data set for 2008 from Enron, SpamAssassin and CEAS according to the original proposal and repeating the analysis using this complete data set, which is a necessary step to treat any of the current accuracy figures as final; (2) implementing the ROC-AUC measure as proposed and a fair SMOTE versus class weighting comparison; (3) fixing the `OUTPUT_FILES` bug mentioned in section 4.2; and (4) running DistilBERT training together with RoBERTa as asked in the original proposal.

6. Bibliography

- [1] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian approach to filtering junk e-mail," in Proc. AAAI Workshop on Learning for Text Categorization, Madison, WI, USA, 1998, pp. 55–62.
- [2] H. Drucker, D. Wu, and V. N. Vapnik, "Support vector machines for spam categorization," IEEE Transactions on Neural Networks, vol. 10, no. 5, pp. 1048–1054, Sep. 1999.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171–4186.

- [4] H. Gascon, C. Uriarte, and K. Rieck, "Exploring the use of text classification in the cyber threat analysis domain," in Proc. ACM Workshop on Artificial Intelligence and Security (AISec), 2017, pp. 13–23.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019.
- [6] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv:1907.11692, 2019.
- [7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.
- [8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [9] I. Jamal et al., "An improved transformer-based model for detecting phishing, spam and ham emails: A large language model approach," Security and Privacy, Wiley, 2024, doi: 10.1002/spy2.402.
- [10] A. Alhuzali, A. Alloqmani, M. Aljabri, and R. Alharbi, "In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets," 2025.